

Module 1

Retrieval-Augmented Generation

Hardeep Johar

Senior Lecturer in the Discipline of
Industrial Engineering and Operations Research

Building Custom LLMs

Human Agent vs LLM Agent - 1

A human agent

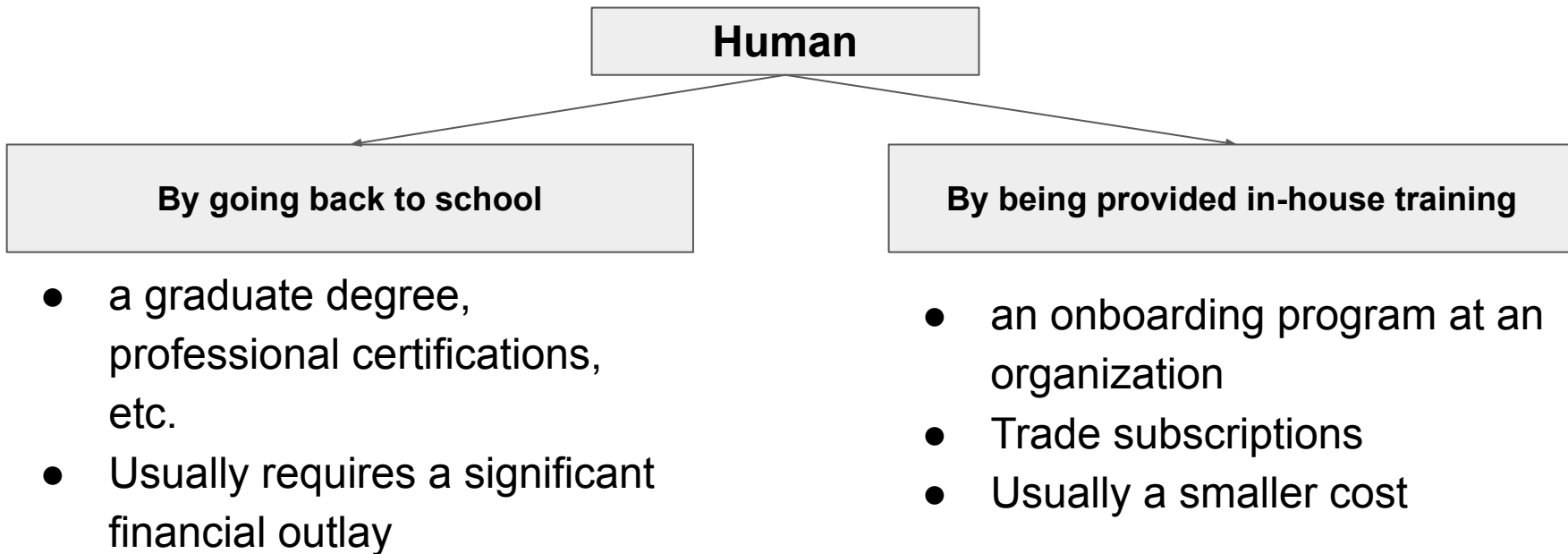
- Has some basic knowledge of the world (e.g., a college degree)
- Can converse in natural language
- Can answer general questions
- Can make deductive inferences
- But doesn't know the internal business knowledge to answer business specific questions

An LLM agent

- Has been trained on large amounts of text data
- Can converse in natural language
- Can answer general questions
- Has some ability to make deductions as long as they have been trained on relevant information
- But doesn't know the internal business knowledge to answer business specific questions

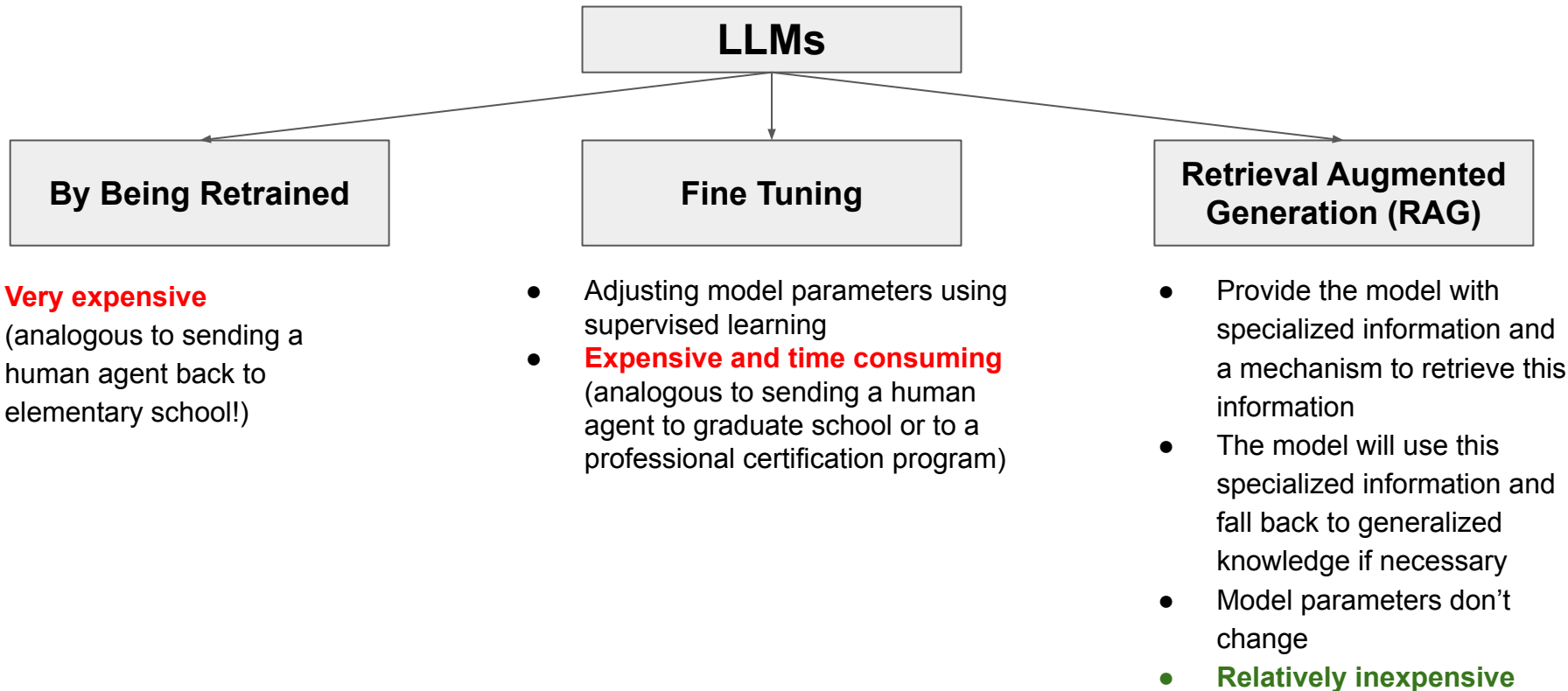
Human Agent vs LLM Agent - 2

- A human agent can add to knowledge



Human Agent vs LLM Agent - 3

- An LLM agent can add to knowledge



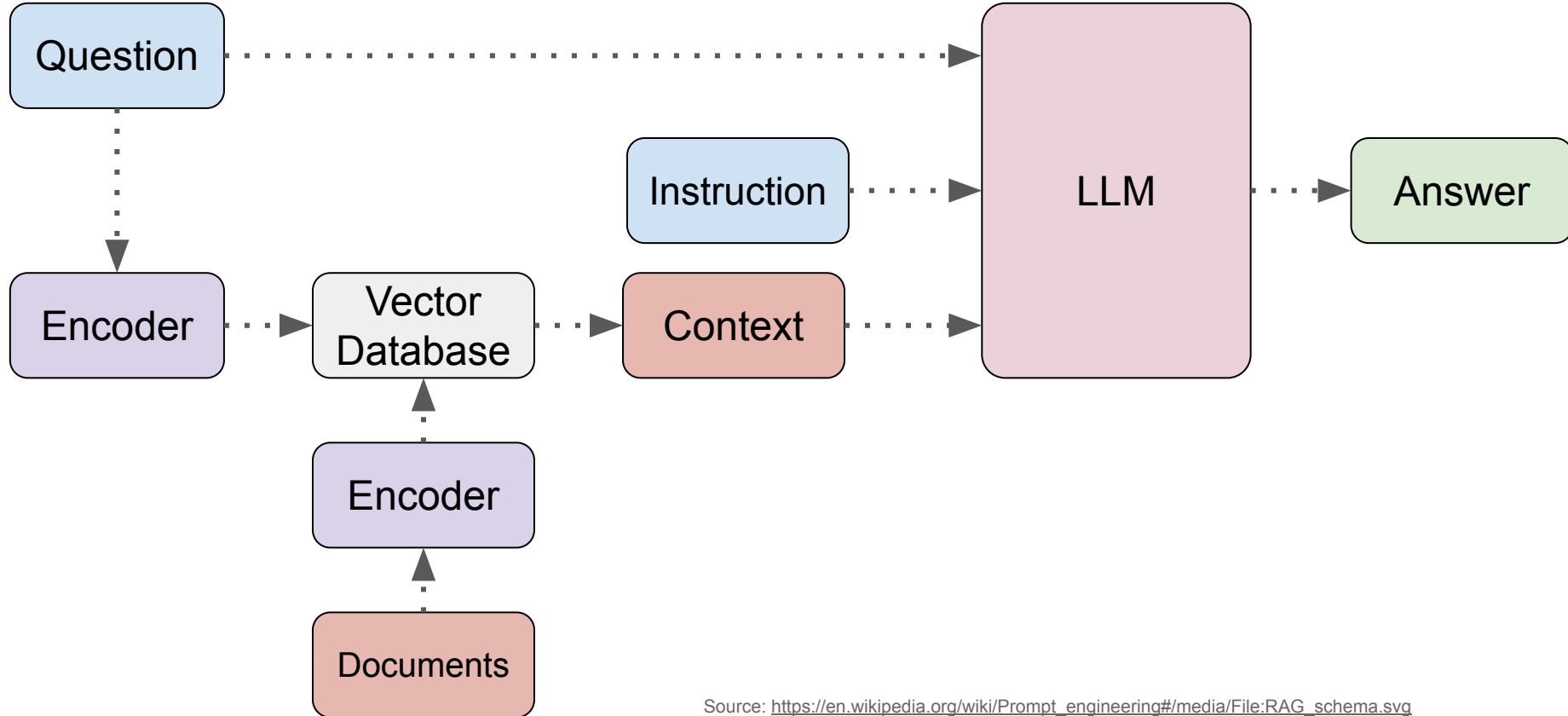
Retrieval-Augmented Generation - 1

- The LLM accesses information from an external (to its model) document repository
- It uses this document repository, as well as its trained model, to answer queries
- Roughly:
 - The documents are converted into chunks (short sequences of words)
 - The chunks are converted into embedded vectors
 - The query is converted into embedded vectors
 - The most similar document embedded vectors are chosen using a similarity algorithm
 - The LLM then uses its “language skills” to respond to the query

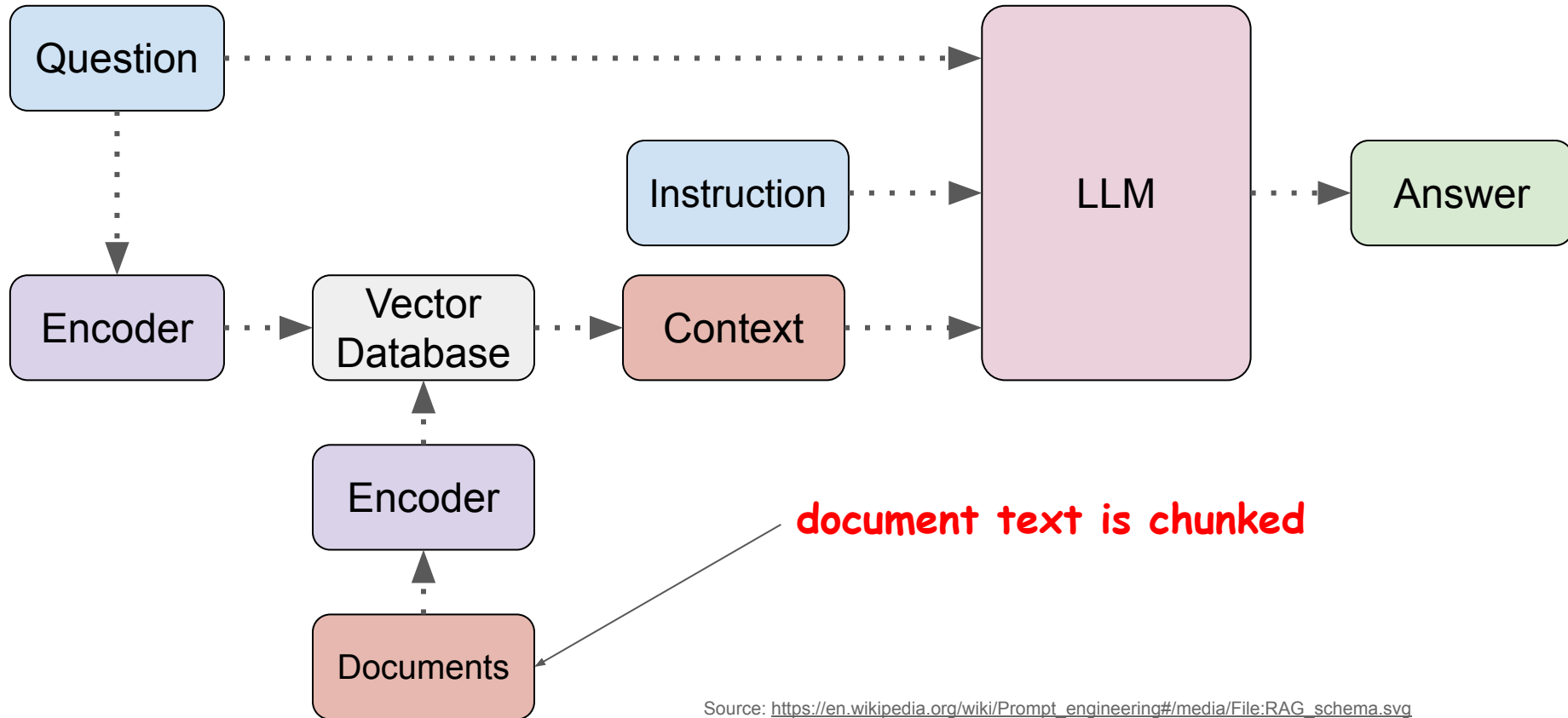
Embedding Chunked Vectors

- Since chunks are smaller groups of information
 - For example, 250 word chunks from a 10,000 word document
 - The likelihood of the “objects” in the chunks being related is high
 - Embedded vectors from these chunks will capture relationships better

Retrieval-Augmented Generation - 2



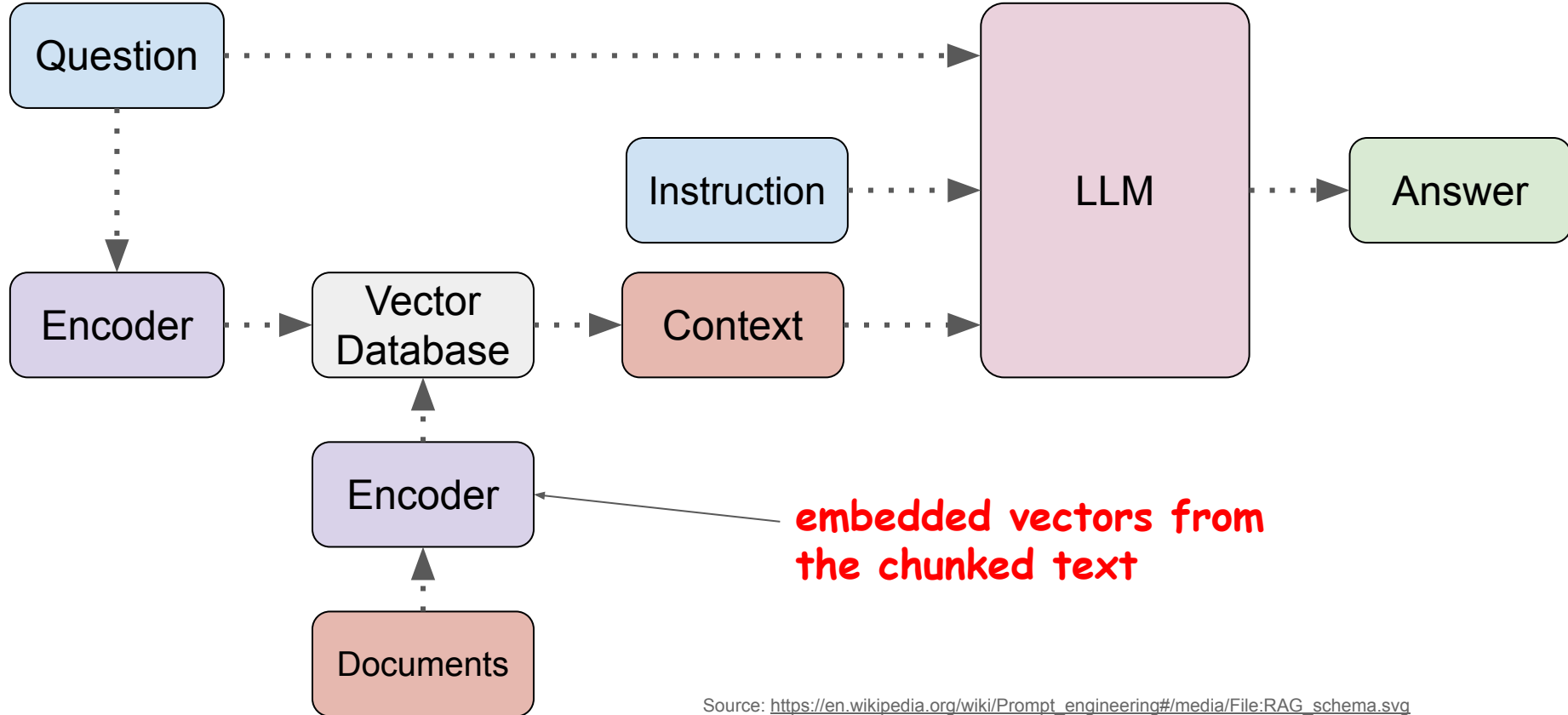
Retrieval-Augmented Generation - 3



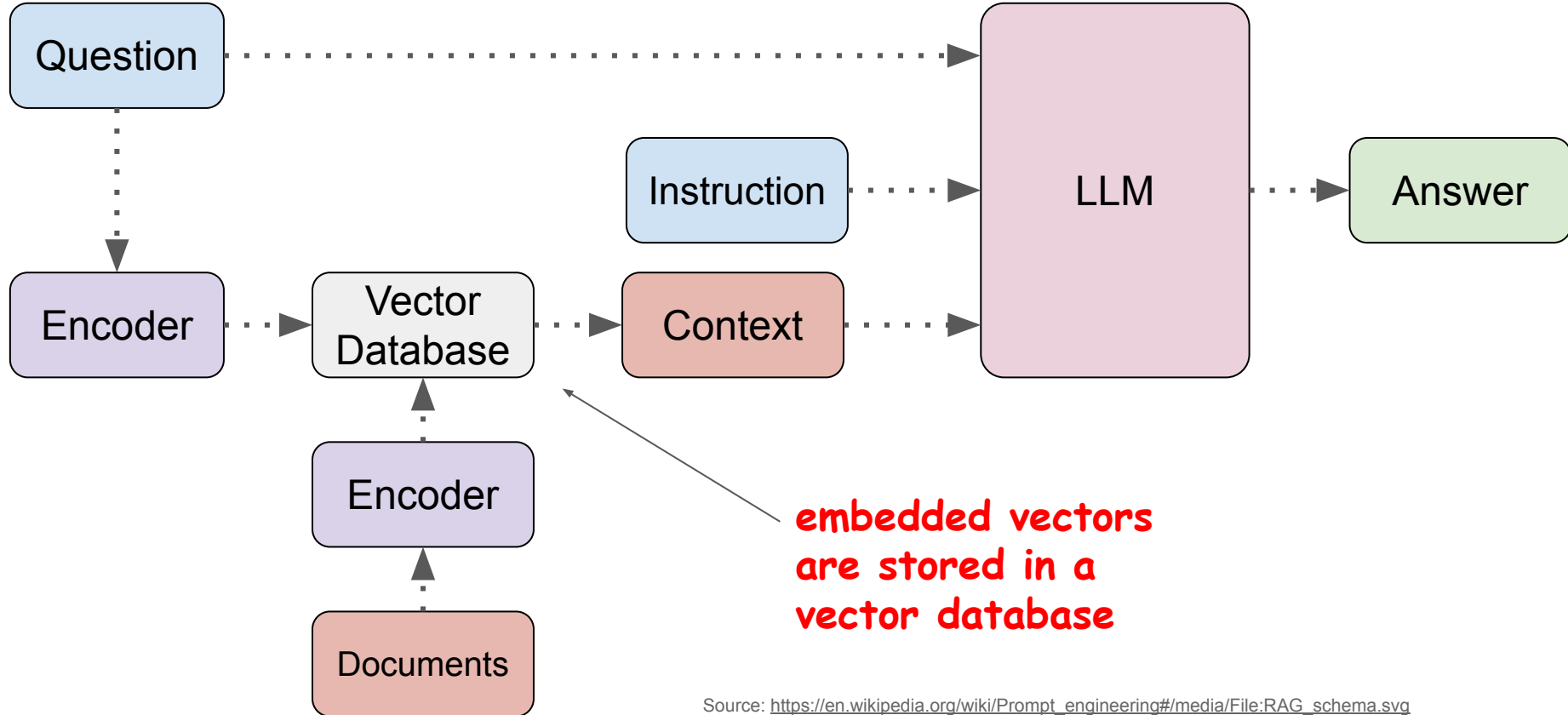
Chunking

- Text documents are large and contain a variety of information
 - A customer service agent may contain information on
 - How to return products
 - With different policies for different products
 - How to contact a human agent
 - Locations of retail stores
- Chunking increases the probability that related information is grouped

Retrieval-Augmented Generation - 4



Retrieval-Augmented Generation - 5



Vector Databases and Vector Search

- Indexed databases for storing vectors
- Given a vector as input, vector databases search for matching vectors using a similarity algorithm
 - Cosine similarity:
 - Calculates the cosine of the angle between two vectors
 - The smaller the angle, the closer the cosine is to 1, and the more similar the vectors

Vector Databases and Vector Search

- The number of vectors is large and calculating the similarity between every pair of vectors is computationally expensive
 - The input prompt is converted into a large number of vectors
 - The document repository is converted into a large number of vectors
 - The product of the two is large

Embedding Vectors and Cosine Similarity Notebook

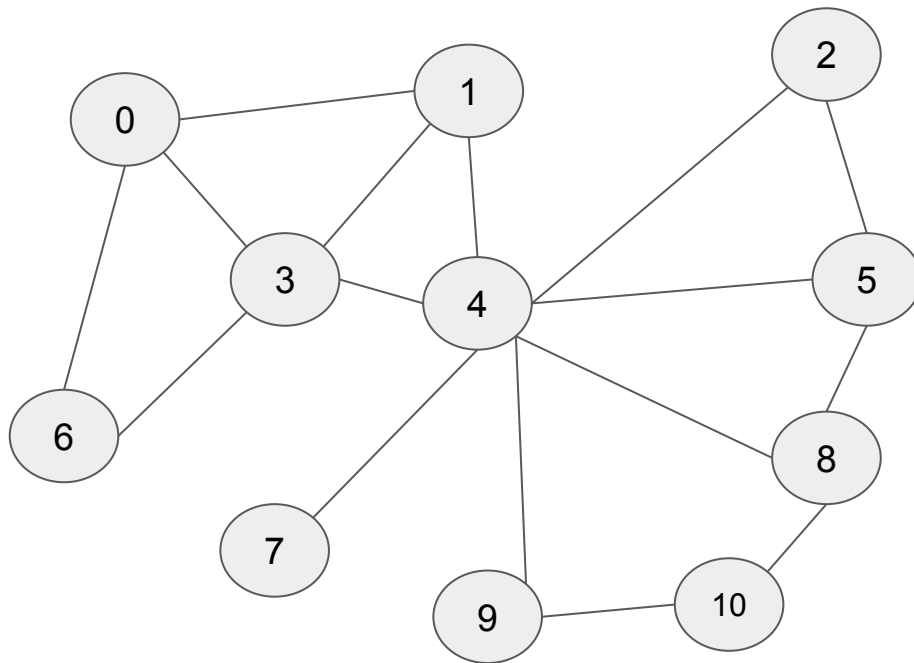
Vector Search: Retrieving Similar Vectors

Navigable Small Worlds (NSW) algorithm

- Store a pre-constructed similarity graph of document chunks
- Randomly pick a chunk and compute the similarity with the input vector
- Move to a neighbor of the chunk and recompute the similarity
- Stop when the similarity doesn't get better
- Repeat and report the top-n similar chunks
- NSW algorithms rely on the “six degrees of separation” idea
 - The best similarity will be utmost some small n away from a random chunk

Navigable Small Worlds - 1

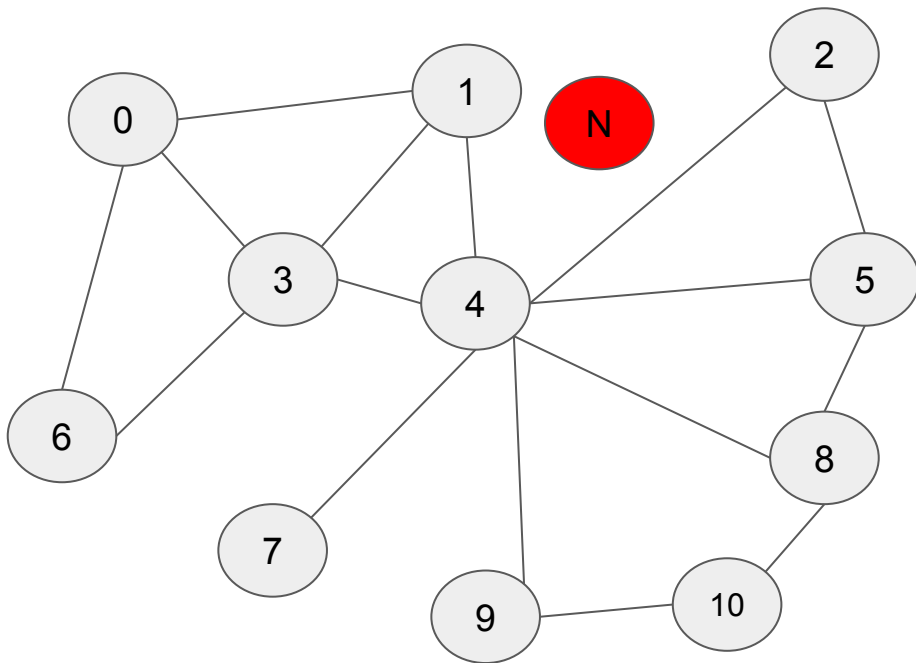
M=2



- Construct the base graph
- The parameter M specifies the number of connections a chunk makes to other chunks
- For a large number of chunks, the graph will be sparse
- The edge attribute is the inverse of cosine similarity

Navigable Small Worlds - 2

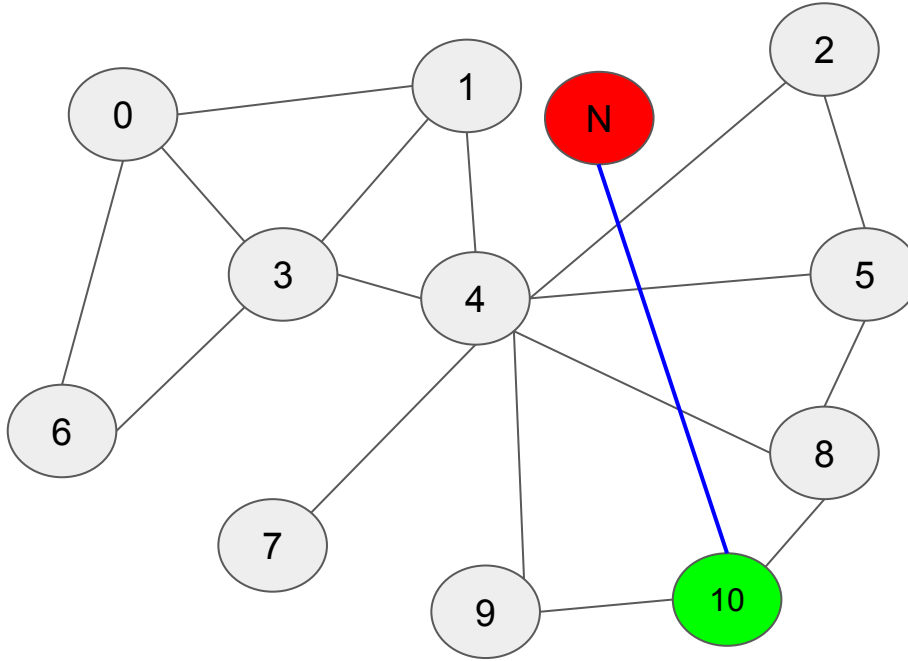
M=2



- A new chunk (from the LLM prompt) arrives
- And is inserted into the graph

Navigable Small Worlds - 3

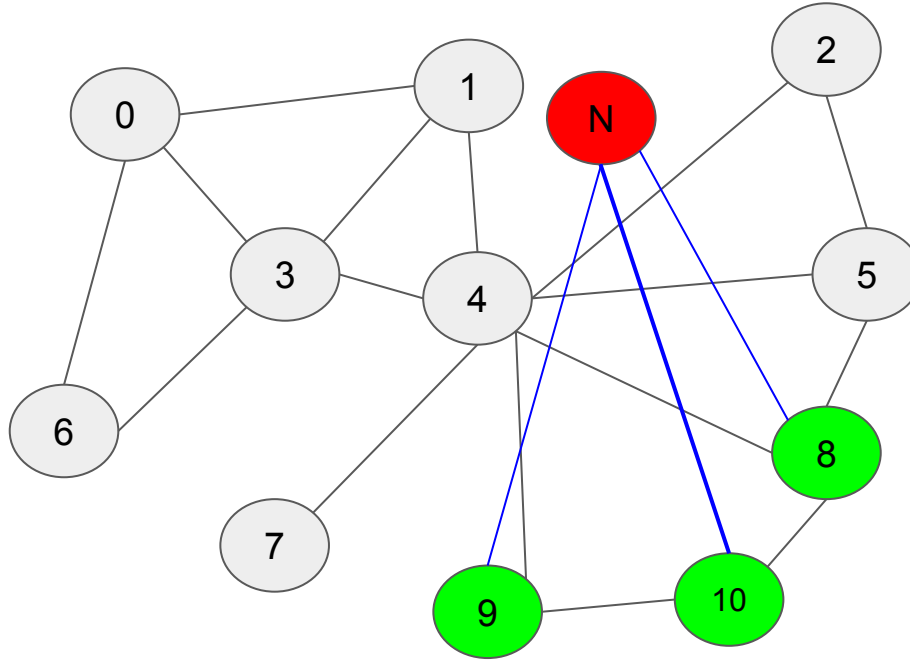
M=2



- Choose a random chunk (e.g., 10)
- And calculate the distance
 - add a new edge between 10 and N

Navigable Small Worlds - 4

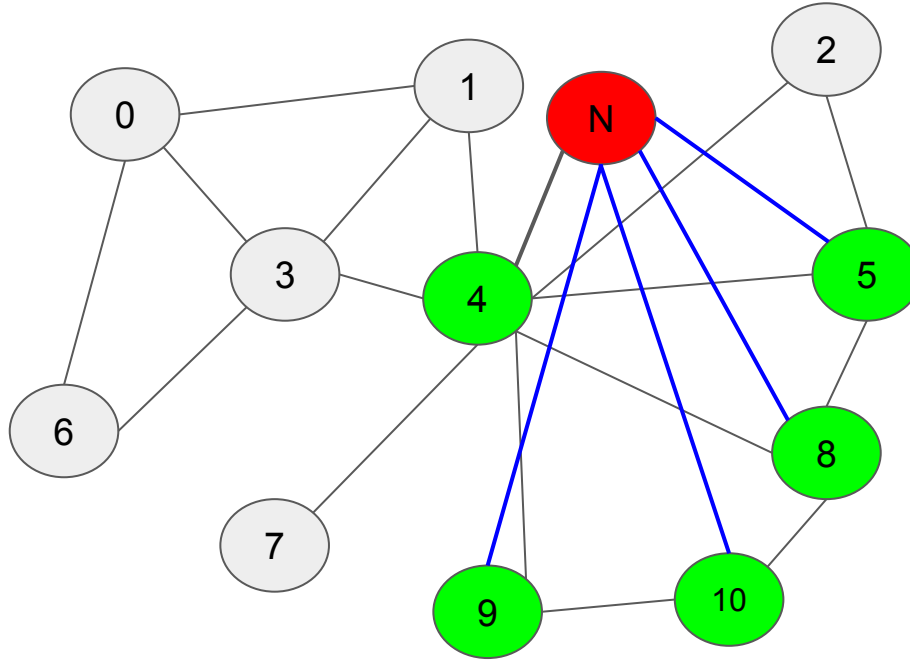
M=2



- Check distance (similarity) from two neighbors
- And calculate the distance
 - add new edges from 9 and 8 to N

Navigable Small Worlds - 5

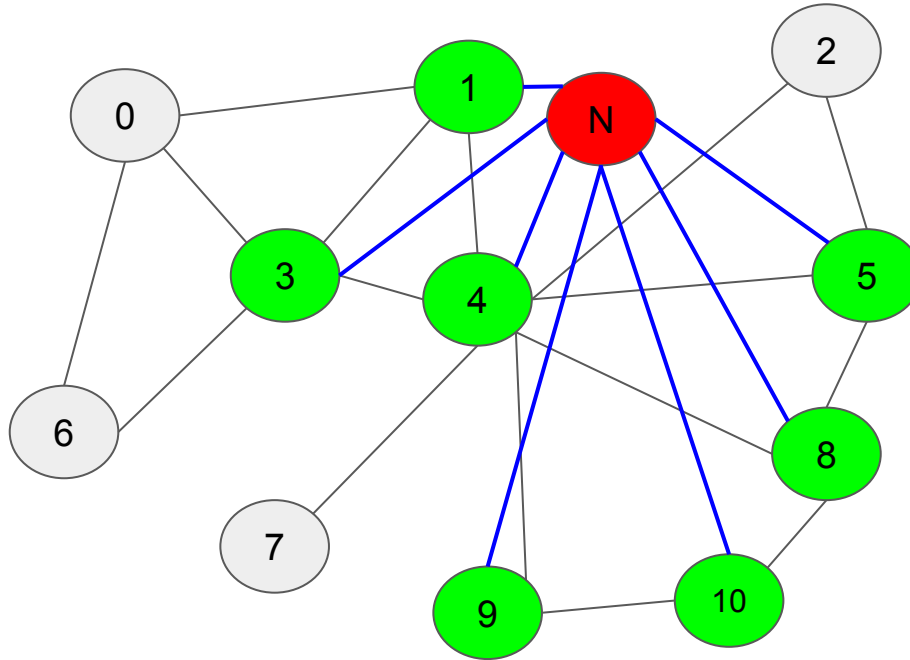
M=2



- 8 is closer
- Calculate distance from 4 and 5 to N

Navigable Small Worlds - 6

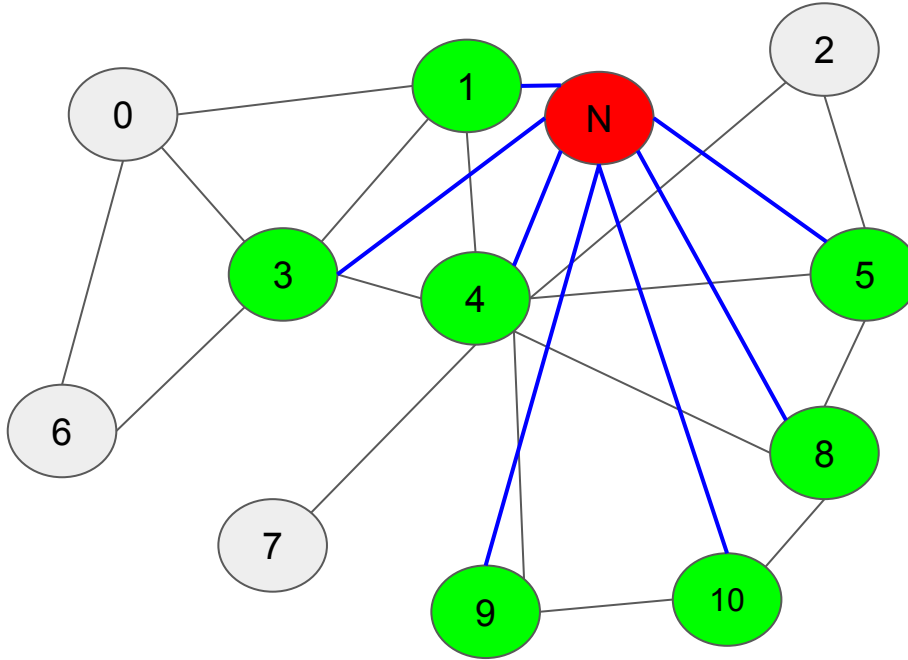
M=2



- 4 is closer
- Calculate distance from 1 and 3 to N

Navigable Small Worlds - 7

M=2

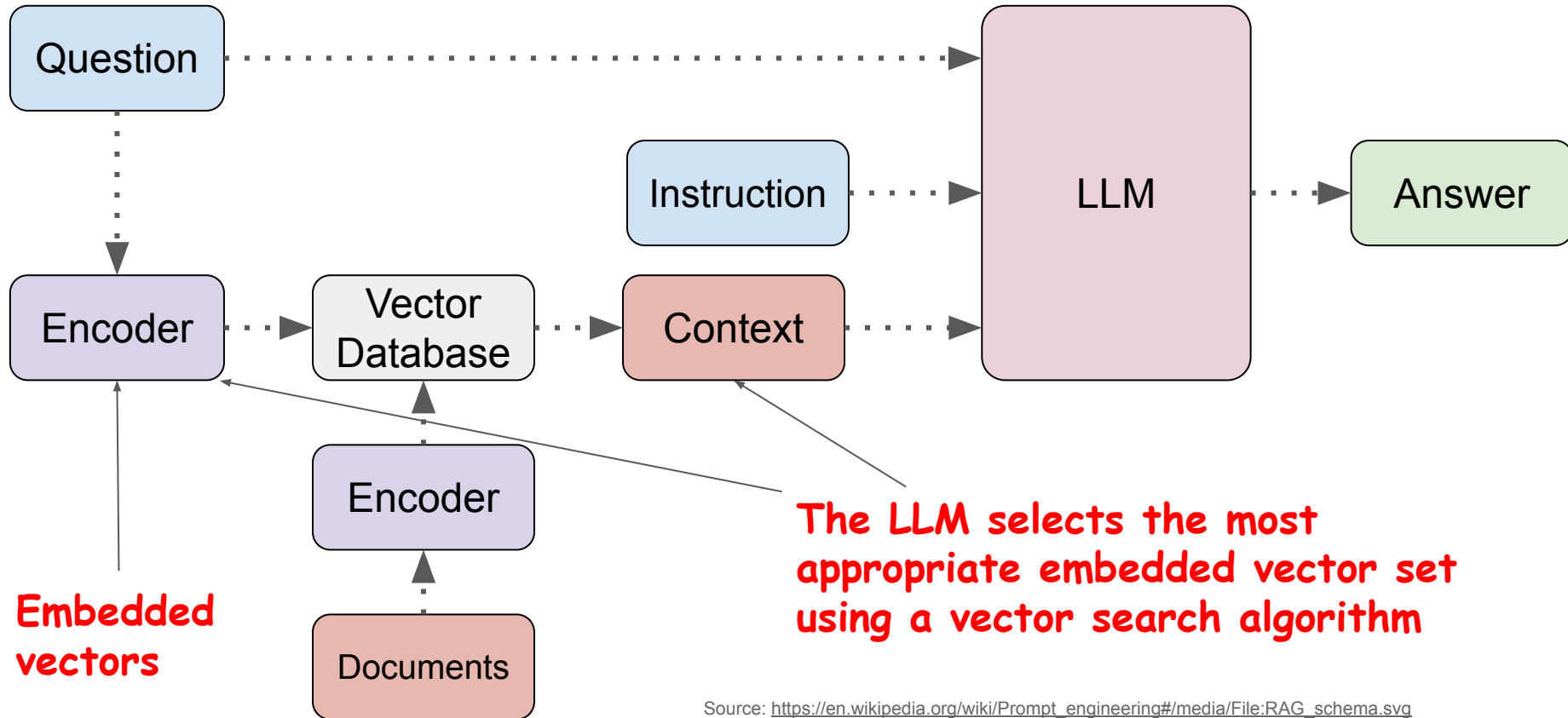


- 1 is closer
- Calculate distance from 0 and 3 to N
- Since neither 0 nor 3 is closer than 1, stop. 1 is the closest chunk

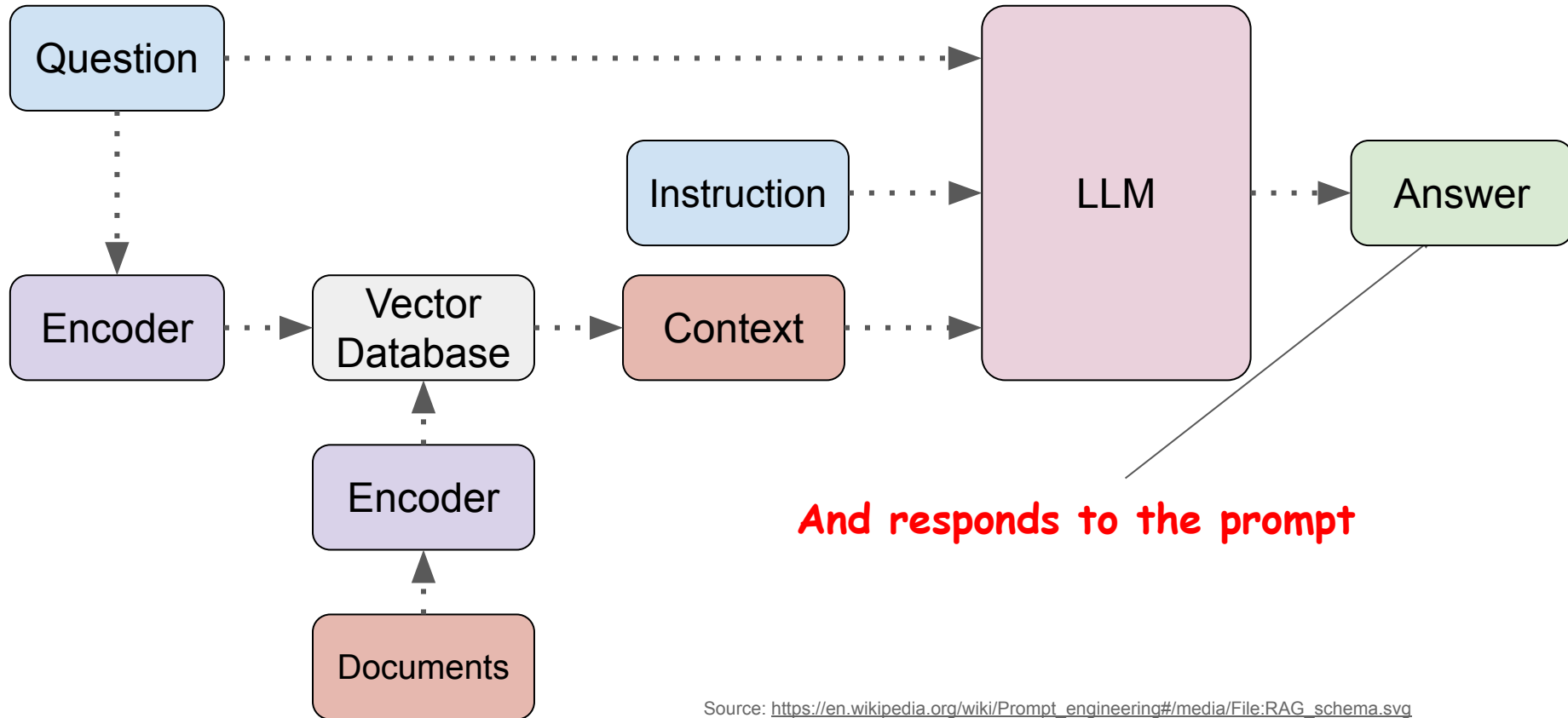
Vector Search: Retrieving Vectors

- Hierarchical Navigable Small Worlds (HNSW) algorithm
 - Adaptation of NSW but the pre-constructed graph is hierarchical with a small number of starting chunks with subsequent chunks arranged in a hierarchy
 - The algorithm picks a random chunk from the top level and then searches in that hierarchy
 - Using the NSW algorithm at each level
 - Facebook AI Search Similarity (FAISS) is a commonly used HNSW implementation and you will see it used later in this course
- Constructing the hierarchy and the base network is the hard part of HNSW
- We won't look at it in detail here but, if you're interested:
 - <https://www.datastax.com/guides/hierarchical-navigable-small-worlds>

Retrieval-Augmented Generation - 6



Retrieval-Augmented Generation - 7



RAG Example Notebook (Xilin)

Knowledge Graphs

- A knowledge graph is a data model that uses a graph to organize and represent domain knowledge in the form of entities and relationships
- Knowledge graphs have a formal semantics for
 - Storing knowledge (entities, relationships)
 - Retrieving knowledge (knowledge searching)

Knowledge Graph: Example